LLD vs HLD in simple words

Think of **HLD and LLD as building a house** 🏠.

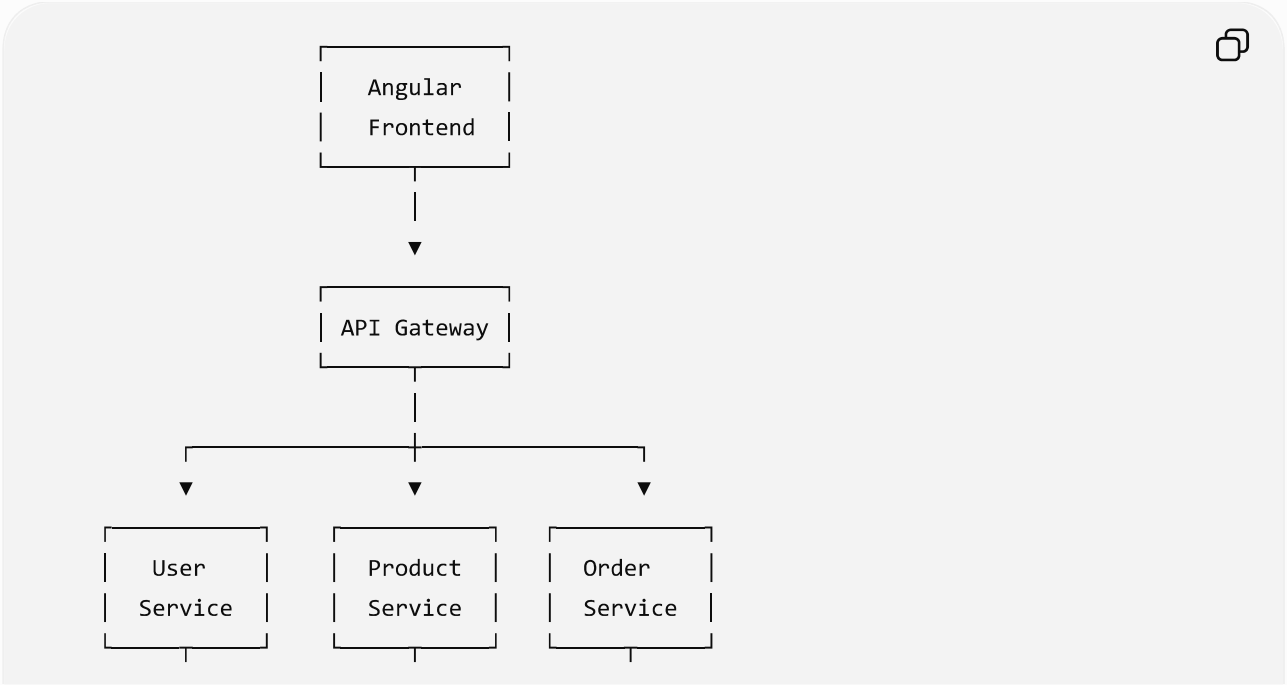
Simple definition

	HLD	LLD
Full form	High-Level Design	Low-Level Design
Focus	**What components do we need?**	**How exactly will each component work?**
Level	Big picture	Detailed implementation
Audience	Architects, senior developers	Developers
Example	API + DB + Redis + Kafka	Classes, methods, DB tables, algorithms

📐 HLD = Big Picture

Imagine you're designing an **e-commerce application**.

HLD says:



```
        |              |              |
        ▼              ▼              ▼
   PostgreSQL     PostgreSQL     PostgreSQL
```

HLD answers questions like:

- What services do we have?
- Which database should we use?
- Do we need Redis?
- Do we need Kafka?
- How do services communicate?
- Where does authentication happen?
- How will the system scale?

HLD = architecture of the entire system.

🔧 LLD = Inside the component

Now take the **Order Service** from the HLD.

LLD goes deeper:

```
OrderController
      |
      ▼
OrderService
      |
      ├── createOrder()
      ├── cancelOrder()
      ├── getOrder()
      |
      ▼
OrderRepository
      |
      ▼
PostgreSQL
```

Then you define things like:

```
Order
----------------
id
userId
totalAmount
```

```
status
createdAt
```

And classes/interfaces:

```
OrderController
OrderService
OrderRepository
PaymentService
InventoryService
```

LLD answers:

- What classes do we need?
- What methods do they have?
- What are the parameters?
- What database tables do we need?
- What relationships exist?
- What design patterns should we use?
- What happens step-by-step when `createOrder()` is called?

LLD = detailed design of individual components.

🎯 One easy way to remember

> **HLD = Google Maps view of the city 🗺️**
> **LLD = Street-by-street navigation 🚗**

Or:

> **HLD:** "We need an Order Service."

> **LLD:** "Here is exactly how `OrderService.createOrder()` works."

Example: Login

HLD:

```
Angular
   ↓
API Gateway
   ↓
```

```
Auth Service
   ↓
PostgreSQL
   ↓
Redis
```

LLD:

```
LoginController
      ↓
AuthService.login()
      ↓
UserRepository.findByEmail()
      ↓
PasswordEncoder.matches()
      ↓
JwtService.generateToken()
      ↓
Return Access Token
```

So, in an interview, a good short answer is:

> **HLD defines the overall system architecture—major components, services, databases, communication, and scalability. LLD defines the internal implementation of those components—classes, methods, database schema, APIs, and design patterns.**

For a **senior developer**, you should be comfortable doing **both HLD and LLD**, because real system-design interviews often move from **"design the system" → "now design this particular service in detail."**

system interview questions & answers of HLD & LLD

Absolutely. If you are preparing for **System Design interviews**, you should prepare HLD and LLD together because interviewers often start with HLD and then drill down into LLD.

Here is a **fresher-to-intermediate interview set**, with answers in simple language.

HLD Interview Questions & Answers

1. What is HLD?

Answer:

HLD stands for **High-Level Design**.

It describes the overall architecture of a system:

- Major components/services
- Database
- APIs
- Cache
- Message queues
- Load balancer
- Communication between services
- Scalability
- Availability

For example:

```
User
  ↓
Angular
  ↓
Load Balancer
  ↓
API Gateway
  ↓
Order Service
  ↓
PostgreSQL
```

HLD focuses on **what components we need and how they communicate**.

---

2. What is LLD?

Answer:

LLD stands for **Low-Level Design**.

It describes the internal implementation of a component.

For example:

```
OrderController
      ↓
```

```
OrderService
      ↓
OrderRepository
      ↓
PostgreSQL
```

We define:

- Classes
- Interfaces
- Methods
- Database tables
- Relationships
- Design patterns
- Business logic

LLD focuses on **how a component will actually be implemented**.

---

3. HLD vs LLD?

HLD	LLD
Big picture	Detailed picture
System architecture	Component design
Services	Classes
Database selection	Tables/schema
API communication	API implementation
Scalability	Business logic
Load balancer	Methods/interfaces

Easy way to remember:

> **HLD = What + Where**
> **LLD = How**

HLD Questions

4. How would you design a URL Shortener like Bitly?

Answer:

Basic architecture:

```
User
 ↓
Load Balancer
 ↓
URL Service
 ↓
 ┌──────────────┐
 │ Redis Cache  │
 └──────────────┘
        ↓
   PostgreSQL
```

When creating a short URL:

```
POST /urls
     ↓
Generate unique ID
     ↓
Store long URL
     ↓
Return short URL
```

When accessing:

```
GET /abc123
     ↓
Redis?
 ↓       ↓
Yes      No
 ↓       ↓
URL    PostgreSQL
 ↓
Redirect
```

Redis improves performance because frequently accessed URLs can be served from memory.

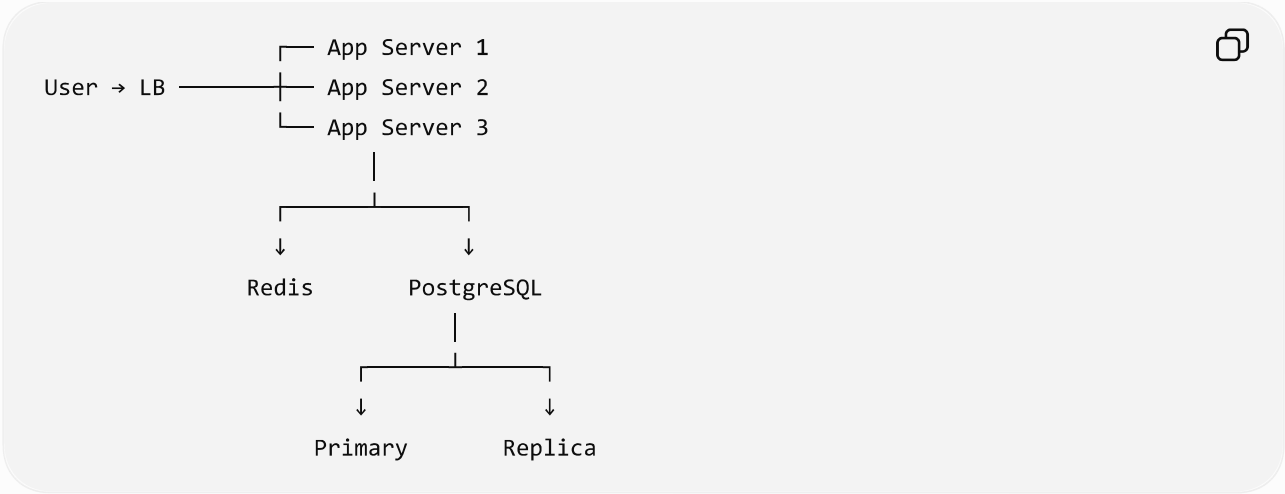
---

5. How would you design a URL shortening system at large scale?

You would consider:

- Load balancers
- Multiple application servers
- Redis
- Database replication
- Database sharding
- CDN where appropriate
- Unique ID generation
- Rate limiting
- Monitoring

Architecture:



---

6. What is horizontal scaling?

Instead of making one server bigger, we add more servers.

```
Before:

User → Server


After:

                ┌→ Server 1
User → LB ──────┼→ Server 2
                └→ Server 3
```

This is called **horizontal scaling**.

It improves capacity and availability.

---

7. Horizontal vs vertical scaling?

Vertical scaling:

```
4 CPU + 8 GB RAM
       ↓
16 CPU + 64 GB RAM
```

Make one machine more powerful.

Horizontal scaling:

```
1 Server
   ↓
5 Servers
```

Add more machines.

For distributed systems, horizontal scaling is generally preferred because it allows better elasticity and fault tolerance.

---

8. What is a load balancer?

A load balancer distributes incoming requests across multiple servers.

```
              ┌→ Server 1
User → LB ──┼→ Server 2
              └→ Server 3
```

Benefits:

- Distributes traffic
- Prevents one server from being overloaded
- Improves availability
- Enables horizontal scaling
- Can perform health checks

---

9. What is caching?

Caching means storing frequently accessed data in a faster storage layer.

Example:

```
Request
   ↓
Redis
   ↓
Cache Hit → Return data
```

If cache misses:

```
Request
   ↓
Redis ❌
   ↓
Database
   ↓
Store in Redis
   ↓
Return data
```

Common caching technology:

Redis

---

10. What is cache-aside?

This is one of the most common caching patterns.

```
Application
    ↓
Check Redis
    ↓
  Found?
  /   \
Yes    No
 |      |
Return  DB
        ↓
      Redis
        ↓
      Return
```

The application manages the cache.

---

11. What is a database index?

An index helps the database find records faster.

Without index:

```
Database
 ↓
Scan many rows
 ↓
Find user
```

With index:

```
Index
 ↓
Find user ID
 ↓
Fetch row
```

Example:

```sql
CREATE INDEX idx_user_email
ON users(email);
```

But indexes also have a cost:

- Extra storage
- Slower INSERT/UPDATE
- Maintenance overhead

---

12. SQL vs NoSQL?

SQL:

Examples:

- PostgreSQL
- MySQL

Good when you need:

- Strong relationships
- Transactions
- Structured data

- Complex queries

NoSQL:

Examples:

- MongoDB
- DynamoDB
- Cassandra

Useful when you need:

- Flexible schema
- Very large scale
- High write throughput
- Specific access patterns

Don't say:

> "NoSQL is always faster."

Instead say:

> "The choice depends on access patterns, consistency requirements, scale and data relationships."

---

13. What is database replication?

Replication means maintaining copies of database data.

```
          Primary
          /     \
         ↓       ↓
Replica 1  Replica 2
```

Primary can handle writes.

Replicas can handle reads.

Benefits:

- High availability
- Read scalability
- Disaster recovery

---

14. What is database sharding?

Sharding means splitting data across multiple databases.

Example:

```
User ID

1-1M       → Shard 1
1M-2M      → Shard 2
2M-3M      → Shard 3
```

Instead of one huge database, data is distributed across multiple databases.

---

15. What is a message queue?

A message queue allows services to communicate asynchronously.

Example:

```
Order Service
     ↓
   Kafka
     ↓
Notification Service
```

Instead of Order Service directly waiting for Notification Service:

```
Order → Kafka → Notification
```

Benefits:

- Decoupling
- Asynchronous processing
- Traffic buffering
- Better resilience

Common technologies:

- Kafka
- RabbitMQ
- AWS SQS

---

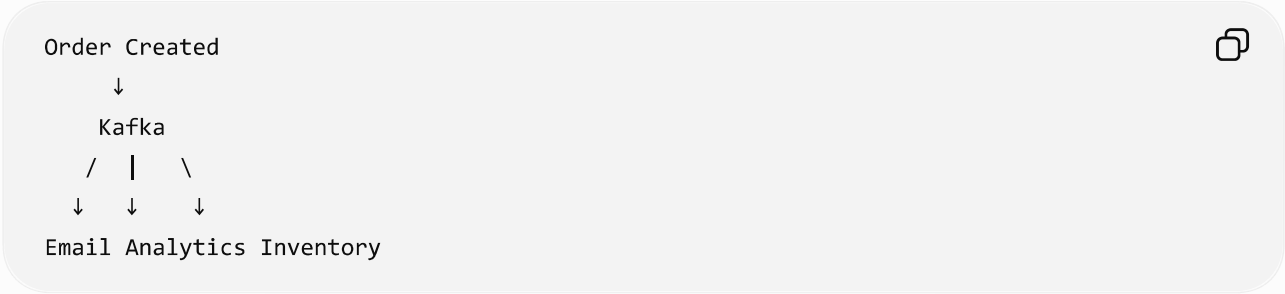
16. Kafka vs RabbitMQ?

A simple interview answer:

RabbitMQ is commonly used for traditional message queuing and task distribution.

Kafka is a distributed event-streaming platform designed for high-throughput event streams, durable logs and replay.

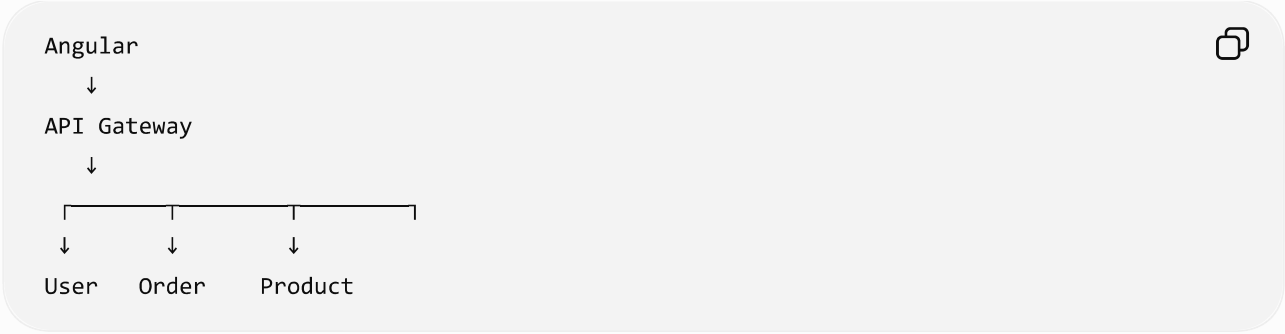
Example:



---

17. What is API Gateway?

API Gateway is an entry point between clients and backend services.



It can handle:

- Routing
- Authentication
- Authorization
- Rate limiting
- Logging
- Request transformation

---

18. What is rate limiting?

Rate limiting controls how many requests a client can make within a period.

Example:

```
100 requests / minute / user
```

If the user sends 101 requests:

```
HTTP 429 Too Many Requests
```

It protects APIs from:

- Abuse
- Accidental overload
- DDoS-like traffic
- Excessive API usage

LLD Interview Questions

19. What is SOLID?

SOLID is a collection of five object-oriented design principles.

```
S → Single Responsibility
O → Open/Closed
L → Liskov Substitution
I → Interface Segregation
D → Dependency Inversion
```

The goal is to create code that is:

- Maintainable
- Testable
- Extensible
- Less tightly coupled

---

20. What is Single Responsibility Principle?

A class should have **one main responsibility**.

Bad:

```
UserService
```

```
createUser()
sendEmail()
generateReport()
saveToDatabase()
```

Too many responsibilities.

Better:

```
UserService
EmailService
ReportService
UserRepository
```

Each has a focused responsibility.

---

21. What is Open/Closed Principle?

Software should be:

> **Open for extension, closed for modification.**

Example:

Instead of modifying payment logic every time:

```
PaymentService
   ├── CreditCardPayment
   ├── UPIPayment
   └── PayPalPayment
```

You can add:

```
BitcoinPayment
```

without heavily modifying existing code.

---

22. What is dependency injection?

Instead of a class creating its own dependency:

```
class OrderService {
    private repo = new OrderRepository();
```

```
}
```

we inject it:

```
class OrderService {
    constructor(private repo: OrderRepository) {}
}
```

Benefits:

- Loose coupling
- Easier testing
- Easier replacement of implementations

This is particularly relevant if you're working with **Angular**, because Angular's dependency injection system is a practical example of this concept.

---

23. What is an interface?

An interface defines a contract.

```
Payment
---------
pay()
refund()
```

Different implementations:

```
CreditCardPayment
UPIPayment
PayPalPayment
```

All follow the same contract.

---

24. What is Strategy Pattern?

Strategy Pattern allows us to choose an algorithm/behavior dynamically.

Example:

```
PaymentStrategy
      |
┌─────┼─────┐
```

```
 ↓    ↓     ↓
UPI  Card  PayPal
```

Example:

```
PaymentService
      ↓
PaymentStrategy
      ↓
UPIStrategy
```

Useful when you have multiple interchangeable algorithms.

---

25. What is Factory Pattern?

Factory creates objects without exposing the creation logic to the client.

Example:

```
PaymentFactory
      ↓
 ┌────┼────┐
 ↓    ↓    ↓
UPI  Card  PayPal
```

Client:

```
factory.create("UPI")
```

The factory decides which object to create.

Important LLD Design Problems

These are extremely common in interviews.

26. Design a Parking Lot

You might have:

```
ParkingLot
 ├── Floor
 │    ├── ParkingSpot
 │    └── ParkingSpot
```

```
│
├── Vehicle
│     ├── Car
│     ├── Bike
│     └── Truck
│
└── Ticket
```

Possible classes:

```
ParkingLot
ParkingFloor
ParkingSpot
Vehicle
Car
Bike
Truck
Ticket
Payment
```

You would discuss:

- Spot allocation
- Vehicle types
- Entry
- Exit
- Ticket generation
- Payment
- Availability

---

27. Design a Library Management System

Classes:

```
Library
Book
BookCopy
Member
Librarian
Loan
Reservation
Fine
```

Flow:

```
Member
  ↓
Search Book
  ↓
Reserve/Borrow
  ↓
Loan
  ↓
Return
  ↓
Calculate Fine
```

---

28. Design an Elevator System

Possible classes:

```
Elevator
ElevatorController
Floor
Request
Button
Door
```

Flow:

```
User
 ↓
Press Floor Button
 ↓
ElevatorController
 ↓
Select Elevator
 ↓
Elevator moves
```

Important discussion:

- How do we select an elevator?
- How do we handle multiple requests?
- What happens when an elevator fails?
- How do we prioritize requests?

---

29. Design a Tic-Tac-Toe game

Classes:

```
Game
Board
Cell
Player
Symbol
Move
```

Flow:

```
Game
 ↓
Player makes Move
 ↓
Board updates
 ↓
Check Winner
 ↓
Next Player
```

The interviewer is testing:

- Object-oriented thinking
- Encapsulation
- Class responsibility
- Extensibility

HLD + LLD Combined Questions

These are especially important for **experienced developers**.

30. Design an Online Ticket Booking System

HLD:

```
User
 ↓
Frontend
 ↓
API Gateway
 ↓
Booking Service
 ↓
 ┌──────────────┐
 │ PostgreSQL   │
```

```
| Redis          |
| Payment        |
| Notification   |
└────────────────┘
```

The critical problem is **preventing double booking**.

Suppose:

```
User A → Seat A1
User B → Seat A1
```

Both try to book simultaneously.

You need concurrency control using mechanisms such as:

- Database transactions
- Row-level locking
- Optimistic locking
- Temporary seat reservation
- Idempotency

---

31. Design a Chat Application

HLD:

```
Angular
   ↓
Load Balancer
   ↓
Chat Servers
   ↓
WebSocket
   ↓
Message Broker
   ↓
Message Database
```

For real-time communication:

```
User A
  ↓
WebSocket
  ↓
Chat Server
```

```
  ↓
Kafka
  ↓
Chat Server
  ↓
WebSocket
  ↓
User B
```

You need to think about:

- Online/offline status
- Message delivery
- Read receipts
- Typing indicators
- Message persistence
- Reconnection
- Ordering
- Scaling WebSocket connections

⭐ Most Important Questions to Prepare

If you're preparing for interviews, I'd prioritize these:

HLD

1. URL Shortener
2. YouTube
3. WhatsApp/Chat
4. Uber
5. Netflix
6. Ticket Booking
7. E-commerce
8. Notification System
9. File Storage System
10. Rate Limiter
11. News Feed
12. Search System

LLD

1. Parking Lot
2. Elevator

3. Library Management
4. Tic-Tac-Toe
5. ATM
6. Vending Machine
7. Splitwise
8. BookMyShow/Ticket Booking
9. Chess
10. Logger
11. Cache
12. Payment System

🧠 How to answer an HLD interview question

Don't immediately start drawing boxes.

Use this sequence:

```
1. Clarify requirements
          ↓
2. Functional requirements
          ↓
3. Non-functional requirements
          ↓
4. Estimate scale
          ↓
5. Define APIs
          ↓
6. High-level architecture
          ↓
7. Database design
          ↓
8. Caching
          ↓
9. Scaling
          ↓
10. Failure handling
```

For example, if asked **"Design WhatsApp"**, don't immediately say:

> "We'll use Kafka, Redis and MongoDB."

First establish **what the system needs to do** and **what scale/constraints you're designing for**.

🧠 How to answer an LLD interview question

Use:

```
1. Identify entities
       ↓
2. Identify responsibilities
       ↓
3. Define classes
       ↓
4. Define interfaces
       ↓
5. Define relationships
       ↓
6. Apply SOLID
       ↓
7. Apply design patterns where useful
       ↓
8. Explain important flows
       ↓
9. Discuss edge cases
```

The most important distinction

When the interviewer asks:

> **"Design an e-commerce system."**

Think **HLD**.

```
Angular
 ↓
API Gateway
 ↓
Microservices
 ↓
Redis
 ↓
Kafka
 ↓
PostgreSQL
```

When they ask:

> **"Now design the Order Service."**

Think **LLD**.

```
OrderController
       ↓
OrderService
       ↓
OrderRepository
       ↓
Order
       ↓
PaymentStrategy
       ↓
InventoryService
```

One sentence to remember for interviews:

> **HLD tells me how the whole system is organized; LLD tells me how the individual components are implemented.**